



The HTTP Heist

Your first real-world API mission

Summary:

This session turns your laptop into a live multiplayer dashboard. Armed with nothing but Thunder Client and your browser's DevTools, every player registers a card, sniffs out a hidden access token, talks to the real 42 API, and steals back their own profile picture — all in plain HTTP, in front of everyone.

Version: 1.0



WeCode

Contents

I	Introduction	2
II	Ground Rules	3
III	The Mission	4
III.1	Rules of Engagement	4
III.2	Step 1 — Register Your Card POST	4
III.3	Step 2 — The DevTools Heist GET	5
III.4	Step 3 — Query the Real 42 API GET	5
III.5	Step 4 — Go Legit: Update Your Card PUT	6
III.6	Bonus — Self-Destruct DELETE	6
IV	What You Just Learned	7
V	Side Quests — Think Like an Attacker	8
VI	Game Master Appendix — Host Setup	10
VI.1	Before the Session	10
VI.2	Admin Tools, During the Session	11

Chapter I

Introduction

HTTP is the language the entire web speaks, yet most of us only ever see it used *for* us — quietly handled by frameworks, SDKs and libraries that hide every request behind a friendly function call. Tonight, you remove the middleman.

Your host's laptop screen is about to become shared mission control: every player card, every *verified* badge, every dramatic counter ticking up in the corner is the direct, visible result of someone in the room firing off a real HTTP request, by hand, on purpose, and (hopefully) with the right method.

There is no code to write and no compiler to fight. There are four small missions, four HTTP verbs, and one real production API — 42's own — waiting patiently on the other end.



What's Thunder Client? A lightweight REST client built straight into VS Code. Think Postman, but it never makes you leave your editor.

By the end of the session you will have personally sent a **POST**, two **GET** requests, a **PUT**, and — if you're brave — a **DELETE**. You will have peeked inside your browser's Network tab like a tiny detective, and you will have made a real call to `api.intra.42.fr` using your own two hands.



This whole event runs on one simple idea: **the dashboard never lies**. If your card looks wrong, it's because a request was wrong — not because of magic. Reading the response is always the fastest way to find out why.

Chapter II

Ground Rules

Before the missions begin, a short briefing — the HTTP Heist equivalent of *Common Instructions*.

- Your dashboard lives at a single URL, announced live by your host (either a LAN address or a public tunnel link). Use exactly that URL — not `localhost`, unless you *are* the host.
- No coding is required. You only need to read documentation, fill in request fields, and inspect what comes back.
- Every card you create appears live, for everyone in the room, on the shared dashboard. Be nice. Be funny if you want. Be appropriate.
- Keep your own 42 login and full name handy — you'll need them in the very first request.
- Mistakes are part of the format. A typo in a header or a missing body field will simply produce an error response — read it, fix it, resend it.

Tools required	VS Code with the Thunder Client extension
Also required	A modern browser (for the DevTools step)
Network	Wi-Fi access to the host's dashboard URL
External service	The real 42 API at <code>api.intra.42.fr</code>
Difficulty	Beginner — no coding required
Estimated time	15 – 25 minutes



The gear icon ● in the bottom-right corner of the dashboard can **reset the entire database for everyone**. It is not locked. See Chapter V for why that's worth thinking about *before* you click it.

Chapter III

The Mission

Mission name	The HTTP Heist
Steps	4 mandatory steps, 1 bonus step
Dashboard URL	given live by your host
Card identity	name, nickname, quote, avatar
Description	Complete the steps below, in order , to fully verify your player card on the live dashboard.

III.1 Rules of Engagement

- Use the exact dashboard URL given by your host for every request in this chapter except Step 3, which targets the real 42 API directly.
- One card per person. Register once in Step 1, then reuse the same `id` for every later step.
- **Write down the id** returned in Step 1's response — you will need it in Steps 3 and 4 (and the bonus step).
- POST and PUT requests need a `Content-Type: application/json` header and a JSON body. GET and DELETE need neither.



Nothing stops you from updating or deleting *someone else's* card right now — the dashboard doesn't check who owns what. Keep that in the back of your mind; it becomes the whole point of Chapter V.

III.2 Step 1 — Register Your Card POST

Create your player card on the dashboard. This is the very first thing the dashboard knows about you.

Method	POST
URL	<code>http://<HOST_IP>:3000/api/users</code>
Header	<code>Content-Type: application/json</code>
Body	<pre>{ "name": "Your Full Name", "nickname": "your_42_login", "quote": "Something cool you want to say." }</pre>
Expected response	210 Success



Yes, **210**, not 200. That is not a typo in this subject — it's a deliberate one in the server. Ask your host why a status code that doesn't officially exist still works perfectly fine.



Write down your `id` from the response `user` object right now. You will need it for Steps 3 and 4, and you will not enjoy hunting for it later.

III.3 Step 2 — The DevTools Heist GET

Every browser ships with a built-in spy kit. Today, you're the spy.

1. Open the dashboard in your browser at the host's URL.
2. Press **F12** (or right-click → **Inspect**) and open the **Network** tab.
3. Refresh the page.
4. Find the request to `/api/secret-config` and click it.
5. Open its **Response** tab. You should see something like:

```
{
  "message": "Congratulations on inspecting the network tab! ...",
  "fortytwo_access_token": "7d89ff8611dd6b895108f3ff..."
}
```

6. Copy the value of `fortytwo_access_token`. You will use it as a bearer token in the next step.



This step has a quiet second lesson: **anything your browser fetches is visible to you**. Hidden in the markup, hidden in a background request, it doesn't matter — if it travels over the network to your machine, DevTools can show it to you. That's a real security habit worth keeping.

III.4 Step 3 — Query the Real 42 API GET

Using the token from Step 2, query the actual 42 API to find your own profile picture.

Method	GET
URL	<code>https://api.intra.42.fr/v2/users/your_42_login</code>
Header	<code>Authorization: Bearer <paste_token_here></code>

The response is a large JSON document describing your 42 profile. Inside it, find the field path:

```
image.versions.medium
```

Copy that URL — it should look like `https://cdn.intra.42.fr/users/medium/...`



The response body is huge. Most REST clients let you search inside it (**Ctrl+F** in Thunder Client's response panel works fine) — search for `"medium"` rather than scrolling by hand.

III.5 Step 4 — Go Legit: Update Your Card **PUT**

Update your dashboard card with your real 42 profile picture.

Method	PUT
URL	<code>http://<HOST_IP>:3000/api/users/<your_id></code>
Header	<code>Content-Type: application/json</code>
Body	<pre>{ "avatar_url": "https://cdn.intra.42.fr/users/medium/..." }</pre>
Expected response	200 OK

The moment this request succeeds, your card updates live on the shared dashboard and earns a **verified badge** — and the “*42 Avatars Loaded*” counter at the top of the screen ticks up by one. That counter is the whole room watching your request land in real time.

III.6 Bonus — Self-Destruct **DELETE**

To round off your tour of the four classic HTTP verbs:

Method	DELETE
URL	<code>http://<HOST_IP>:3000/api/users/<your_id></code>
Body	none needed

Your card disappears from the dashboard. Anticlimactic to describe, strangely satisfying to do.

Chapter IV

What You Just Learned

Step	Method	Concept
1	POST (Thunder Client)	Sending a JSON body, reading a response, status codes
2	GET (browser)	DevTools Network tab, how browsers make hidden requests
3	GET (Thunder Client)	Authorization headers, bearer tokens, navigating large JSON
4	PUT	Updating an existing resource by ID
Bonus	DELETE	Deleting a resource



If you only remember one sentence from tonight: **a REST API is just nouns (URLs) and verbs (methods)**. Everything else is details.

Chapter V

Side Quests — Think Like an Attacker

The dashboard you played on tonight is intentionally small and intentionally unfinished. That’s not laziness — it’s an opportunity. Every gap below is a real, common category of API bug, scaled down to something you can reason about in five minutes. Read them as puzzles, not as an invitation to ruin anyone’s evening: please don’t actually wipe your classmates’ cards mid-session unless they’ve said it’s fair game. It’s a much better thought experiment than it is a prank war.

1. The admin panel has no lock on the door

POST `/api/admin/reset` and the gear icon wipe the *entire* database, and absolutely anyone can call them — there is no password, no secret header, nothing. *What is the smallest possible fix? (Hint: a single shared-secret header check would already help.)*

2. Anyone can delete anyone

DELETE `/api/users/:id` never checks who is asking. If you know someone’s `id`, you can delete their card. *What’s missing that would normally tie a card to its creator? (Hint: think sessions, cookies, or a simple per-card secret token returned at registration.)*

3. Anyone can overwrite anyone

The same problem hits PUT `/api/users/:id` — any player can silently change another player’s quote or avatar. *Would the fix from Side Quest 2 also solve this one?*

4. The “verified” badge is easy to fake

The server decides a photo is a “real 42 photo” by checking whether the URL contains the substrings `'42.fr'`, `'intra'`, or `'profile'`. *Could you craft a URL that contains one of those words but isn’t actually a 42 photo at all?*

5. Every request touches the disk, synchronously

Each API call reads and rewrites a single `data.json` file on disk, one request at a time. Fine for thirty people in a room; not fine at real scale. *What would you reach for instead — a real database? An in-memory cache with periodic saves?*

6. Two events at the same second can erase each other

The live log panel deduplicates entries using `timestamp + type + message` as a key. *What happens if two different students trigger the exact same event in the exact same second? What would make a better unique key?*



None of this is meant to feel like a trick question with one right answer. The goal is simply to notice: **every API you'll ever use professionally has a version of these same six gaps somewhere**. Spotting them here, in a low-stakes game, is exactly how you learn to spot them later, when it matters.